



Results

0%

of text is likely AI ⓘ



AI-generated ⓘ 0%

Human-written & AI-refined ⓘ 0%

Human-written ⓘ 100%



Caution: Our AI Detector is advanced, but no detectors are 100% reliable, no matter what their accuracy scores claim. Never use AI detection alone to make decisions that could impact a person's career or academic standing.

1. Introduction

This assignment will involve finding, exploiting, and fixing the memory corruption flaw present within the C code. This project will provide an insight into how a vulnerability arises through insecure coding, how that vulnerability can be exploited, and how best to address this problem via secure coding methods. The memory corruption flaw covered in this assignment is buffer overflow attacks, which have been known for a long time as among the most serious and important dangers that software developers are facing because of their potential effects on memory manipulation and execution flow.

2. Development Environment Setup

The program was created and debugged through the use of Visual Studio Code as the main text editor, GCC as the compiler, GDB for debugging, and finally MSYS2 as the platform. The first step was to install the C/C++ extension in Visual Studio Code in order to provide the ability to compile and debug. MSYS2 was then installed on the computer, and added to the PATH to ensure that GNU commands can be run from within the Windows command line. Once everything had been set up properly, GCC was checked to confirm the successful installation of C compilers.

3. Vulnerable Program Implementation

Figure 1: vulnerable.c code

4. Vulnerability Analysis

The program contains a stack-based buffer overflow vulnerability caused by the use of the `gets()` function. This is regarded as inherently insecure because there are no limits applied to the boundaries of the user's inputs. This means that the user will be allowed to enter more data than can be stored by the available buffer size, in this instance, 10 bytes. If the input surpasses the specified limit, the extra characters overwrite other data in the stack memory space. This results in unexpected changes in other variables like the "secret" variable, or even worse, application crashes and code execution. This is because the reason behind this vulnerability is the lack of any kind of input validation by the system, and the use of an insecure library function.

5. Compilation with Disabled Protections

The vulnerable code was built via the command below:

```
gcc -g -O0 -fno-stack-protector -o vulnerable vulnerable.c
```

The above compilation options were carefully selected to aid in exploiting the program. The use of the `-g` option makes it possible to generate debugging information, making it easier to analyse the program flow in the debugger. The `-O0` option disables any form of code optimization by the compiler. `-fno-stack-protector` tells the compiler not to apply any stack protection methods at all, like using stack canaries, thus making it simpler for exploitation through buffer overflow. The following example represents an insecure system without any security defenses.

6. Exploitation Process

The compiled code was run by using `./vulnerable`, with an intentionally large input data, which consists of repeated characters.

Figure 2: Exploited input and output

This output demonstrates that the program behaved unexpectedly, as the value of the secret variable was altered from its original value.

7. Explanation of Exploit

Although the allocation of buffer space in the program is only 10 bytes, the size of the input data is relatively very big in comparison with the allocated memory space. As there is no check for buffer memory space, any additional data after the buffer memory will be written into some other part of the stack memory space. As the secret value lies close to the buffer in the memory space, it gets overwritten by this extra information. Initially, the value of secret is set to 12345; however, after the overflow occurs, it is replaced with an arbitrary integer value derived from the overwritten memory. This is proof enough that there has been memory corruption and the hacker has obtained unexpected control of data in the program.

8. Debugging and Memory Inspection

To further study this vulnerability, the application was tested using GDB. The first step involved running the program using GDB and then inserting a breakpoint right at the beginning of the main() function. This way, we make sure that the program does not execute any instructions before we have the chance to verify our variables' values. Once the code was executed step-by-step, it turned out that the value of the secret variable is 12345 prior to any user input processing. Following the execution, when the input buffer is flooded, the segmentation fault (SIGSEGV) occurs. A crash has occurred, which clearly indicates that the stack was damaged due to the buffer overflow.

Figure 3: Secret variable before the input

Figure 4: GDB output after entering the input

9.

Secure Coding Improvements

One of the changes introduced in the safe version of the program is the replacement of the unsafe gets() function with the fgets() function. Unlike gets(), fgets() checks the boundary condition by making sure that the number of characters to be stored in the buffer never exceeds its limit. The validation of user inputs is achieved via fgets(). By removing the new line character from the string by using strcspn(), the input data would have been properly formatted. This way, we guarantee that there will be no buffer overflow.

11. Secure Program Output

Figure 6: Output for secure.c

The output verifies that the input is securely truncated to comply with the buffer constraints, and at the same time, the secret variable does not change in any way. The vulnerability has thus been effectively addressed.

Conclusion

It is clear from this assignment that knowledge has been gained about how to exploit vulnerabilities in the memory system and also to mitigate against such attacks using appropriate countermeasures. Poor programming had some very serious negative impacts that could be seen firsthand. The shift from insecure code to secure code highlights the importance of proper input handling and defensive programming. Generally speaking, the practical exercise illustrates the problems that arise in cybersecurity in the real world and the need for secure coding techniques.

14. Further Security Recommendations

In addition to all of the above solutions, there are other security mechanisms that can be used to

enhance the quality of the code and make sure that it is written using secure programming techniques. Compiler flags like stack canaries using `-fstack-protector` can be employed to detect any form of stack corruption, whereas techniques like ASLR almost eliminate the chances of exploiting the program because of the random allocation of memory addresses. Beyond code-level improvements, security should be integrated throughout the software development lifecycle. The use of automated software such as Cppcheck and Clang Static Analyzer could be utilized via automatic pipeline creation for discovering weaknesses at an early development phase. The whole process could even become more advanced by incorporating the strategy of continuous integration, where automatic discovery of weaknesses is done using software such as Semgrep or SonarQube. Moreover, it is vital to follow threat modeling processes like STRIDE, carry out dynamic tests by using various software packages like OWASP ZAP and Burp Suite, as well as maintain transparency through the use of Software Bill of Materials (SBOM). Through such measures, the organization can be guaranteed that not only will the vulnerabilities be remediated, but they will be prevented from occurring altogether.